

LabVIEW 接口开发说明书

本手册中所提及的其它软硬件产品的商标与名称，都属于相应公司所有。

本手册的版权属于中国大恒（集团）有限公司北京图像视觉技术分公司所有。未得到本公司的正式许可，任何组织或个人均不得以任何手段和形式对本手册内容进行复制或传播。

本手册的内容若有任何修改，恕不另行通知。

© 2020 中国大恒（集团）有限公司北京图像视觉技术分公司版权所有

网 站：<http://www.daheng-imaging.com>

销售信箱：sales@daheng-imaging.com

销售热线：010-82828878 转 8068

支持信箱：support@daheng-imaging.com

支持热线：400-999-7595

目录

1. 相机工作流程	1
1.1. 整体工作流程	1
1.2. 功能控制流程	2
2. 编程指引	3
2.1. 搭建编程环境	3
2.1.1. LabVIEW VI 介绍	3
2.1.2. LabVIEW 程序开发	3
2.2. 快速上手	5
2.2.1. 初始化和反初始化	5
2.2.2. 枚举设备	5
2.2.3. 打开关闭设备	5
2.2.4. 采集控制	5
2.2.5. 相机控制	6
2.2.5.1. 属性控制器	6
2.2.5.2. 属性数据类型	6
2.2.6. 错误处理	9
3. 模块接口定义	10
3.1. InitLib.vi	10
3.2. CloseLib.vi	10
3.3. UpdateAllDeviceList.vi	10
3.4. OpenDeviceByIndex.vi	10
3.5. CloseDevice.vi	11
3.6. IsImplemented.vi	11
3.7. GetBoolValue.vi	11
3.8. SetBoolValue.vi	12
3.9. GetIntValue.vi	12
3.10. SetIntValue.vi	13
3.11. GetFloatValue.vi	13
3.12. SetFloatValue.vi	13
3.13. GetEnumValue.vi	14
3.14. SetEnumValueByString.vi	14
3.15. GetStringValue.vi	15

3.16. SetStringValue.vi	15
3.17. SetCommandValue.vi	15
3.18. StartGrab.vi	16
3.19. StopGrab.vi	16
3.20. GrabImageDisplay.vi	16
3.21. DQBuf.vi	17
3.22. QBuf.vi	17
3.23. ExportConfigFile.vi	18
3.24. ImportConfigFile.vi	18
3.25. GetNodeAccessMode.vi	18
3.26. ImageFormatConvert.vi	19
3.27. GetDataStreamHandleFromDev.vi	20
3.28. FlushQueue.vi	20
3.29. GetOptimalPacketSize.vi	20
3.30. GetLastError.vi	21
3.31. GetInterfaceHandle.vi	21
3.32. GetU3VDeviceInfo.vi	21
3.1. GetGEVDeviceInfo.vi	22
3.1. GetUSBDeviceInfo.vi	22
3.1. GetCXPDeviceInfo.vi	22
4. 常见问题解答	23
5. 版本说明	24

1. 相机工作流程

1.1. 整体工作流程

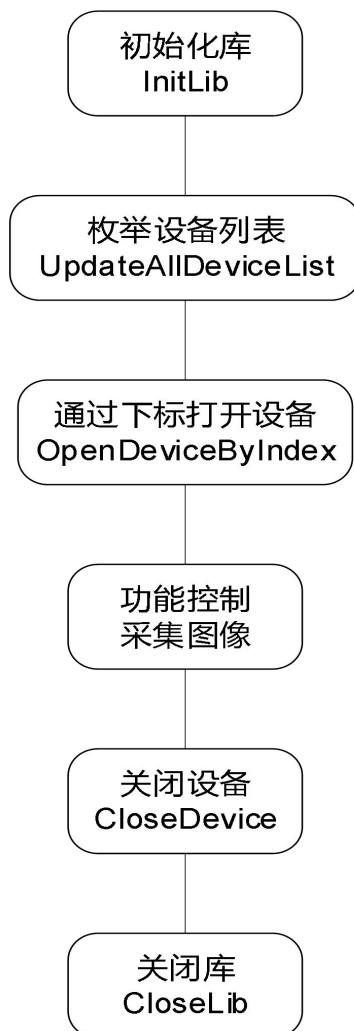


图 1-1 整体工作流程

1.2. 功能控制流程

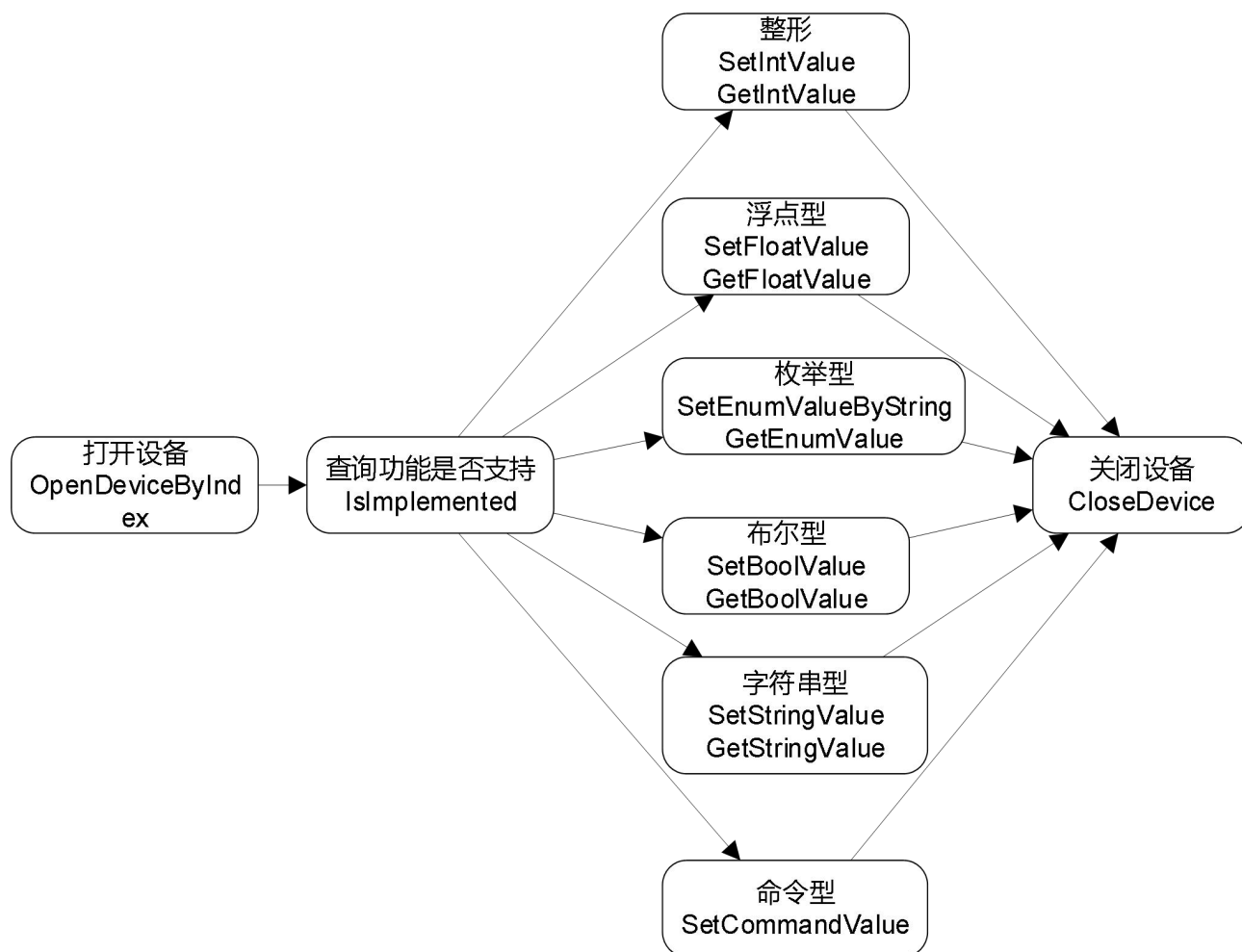


图 1-2 功能控制流程

2. 编程指引

2.1. 搭建编程环境

支持 LabVIEW2016 及以上版本,使用时除安装 LabVIEW 程序外还需安装对应版本的 VISION 视觉库。

2.1.1. LabVIEW VI 介绍

为了方便用户在 LabVIEW 平台调用 GxI API 接口的 SDK,我们在 LabVIEW 平台封装了子 VI。在 GxLVLlib.lvlib 中,一共有 37 个子 VI。这些 VI 是对部分 GxI API 接口的 SDK 的二次封装,其内部调用了 GxI API 接口的 SDK 的一些动态链接库,用户也可以在 LabVIEW 中直接调用 GxI API 接口。

2.1.2. LabVIEW 程序开发

首先新建一个项目,在项目管理器中,右击**我的电脑 (My Computer)**,选择**添加»文件**,将我们的 GxLVLlib.lvlib 添加到项目中。新建一个 VI,命名为 Demo.vi。然后打开 Demo.vi,在其程序框图面板就可以添加 GxLVLlib.lvlib 中的 VI 进行相机操作。

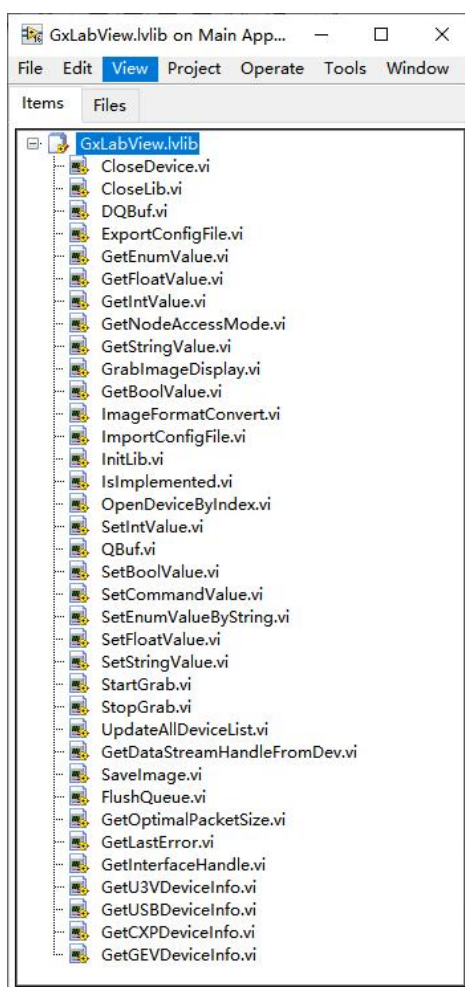


图 2-1 VI 接口列表

注意：LabVIEW 在加载 VI 初始化过程中，可能会出现图 2-2 界面：

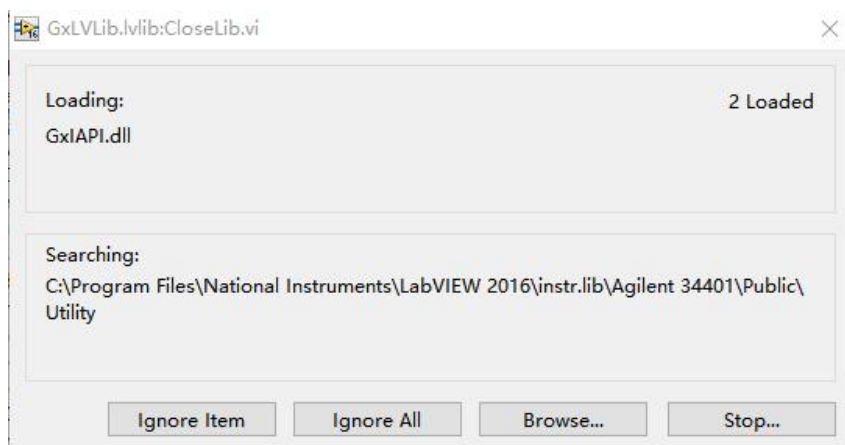


图 2-2 加载 VI 界面

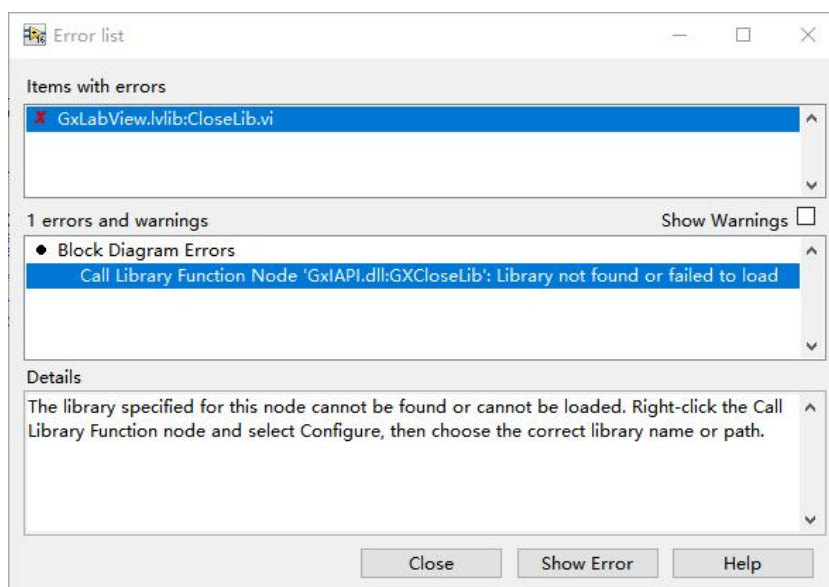


图 2-3 报错界面

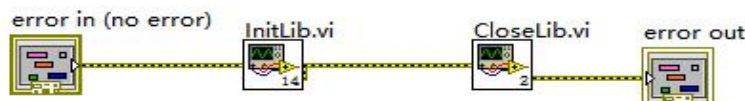
图 2-2 该界面会自动搜索 VI 调用到的 GxIAPL.dll，如果无法搜索到，请用户手动点击浏览添加 SDK 中的 GxIAPL.dll 路径。图 2-3 该界面出现时，将 GxIAPL.dll 放到环境变量 path 可以访问的路径，放到 GxIAPL.dll 安装包路径位置即可。

2.2. 快速上手

2.2.1. 初始化和反初始化

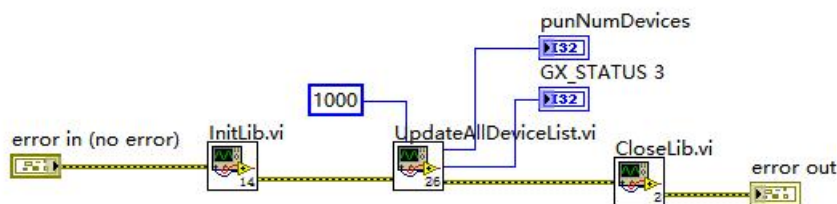
GxLVLlib.lvlib 为 GxI API 接口库的二次封装，而 GxI API 在使用时必须执行初始化和反初始化。

在调用其他接口之前必须调用 InitLib.vi 执行初始化，在进程退出之前必须调用 CloseLib.vi 释放申请的所有资源。



2.2.2. 枚举设备

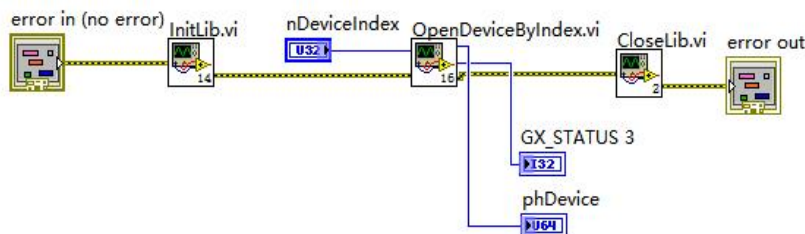
用户通过调用 UpdateAllDeviceList.vi 枚举当前所有可用设备，函数返回值为在线相机的个数。元素的数据类型为 int。



2.2.3. 打开关闭设备

用户可通过 OpenDeviceByIndex.vi 打开设备，函数的输入参数为设备序列号，类型为 int。

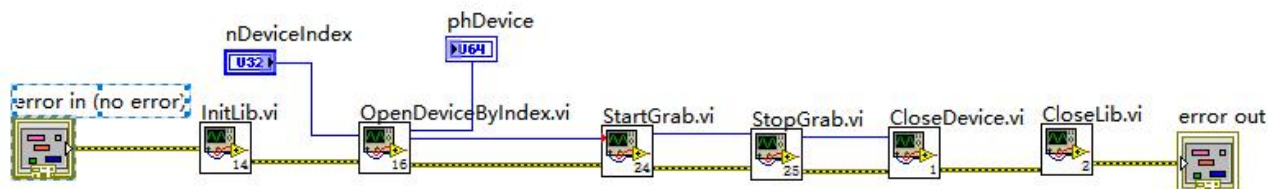
用户在打开设备前应先调用枚举设备接口，更新 GxI API 库内部的设备列表，否则可能打开设备失败。用户可以调用 CloseDevice.vi 来关闭设备，释放所有设备资源。



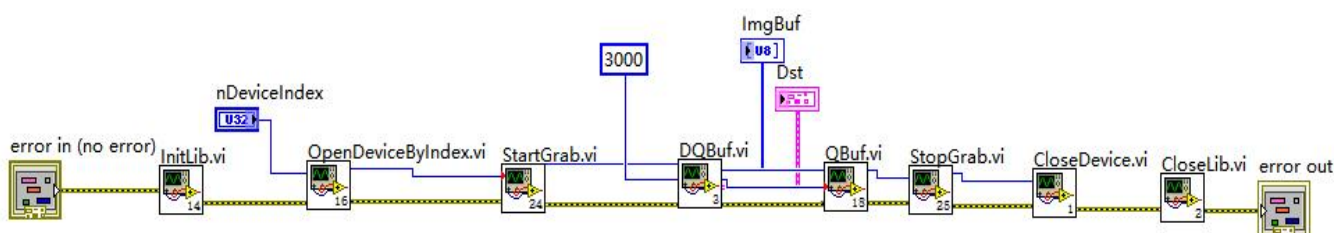
2.2.4. 采集控制

在设备正常开启并设置好相机采集参数后，用户可调用 StartGrab.vi 和 StopGrab.vi 执行开停采。

执行开采时，需要先打开流通道数据流，目前仅支持第 0 通道数据流，然后执行 StartGrab.vi 即可。执行停采时，需要先执行 StopGrab.vi，然后关闭流通道。



用户可通过 DQBuf 和 QBuf 方式获取图像和归还图像。



2.2.5. 相机控制

2.2.5.1. 属性控制器

远端设备属性控制器，比如宽高、曝光、增益等。我们提供的 VI 可实现远端设备属性控制器的访问和控制。

2.2.5.2. 属性数据类型

VI 接口中共实现了 6 种数据类型，如下：

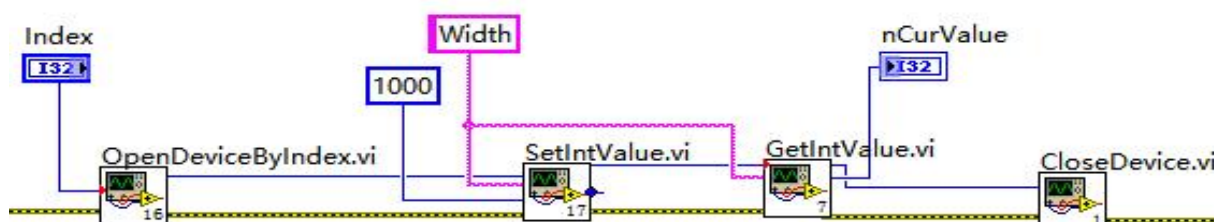
整型：

相关接口：

SetIntValue.vi //设置

GetIntValue.vi //读取

代码样例：



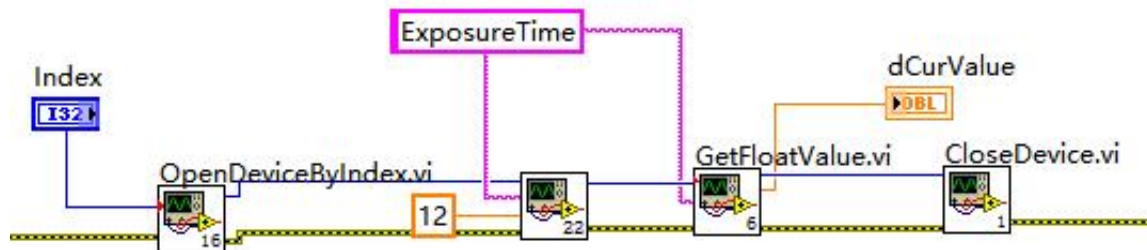
浮点型：

相关接口：

SetFloatValue.vi //设置

GetFloatValue.vi //读取

代码样例：



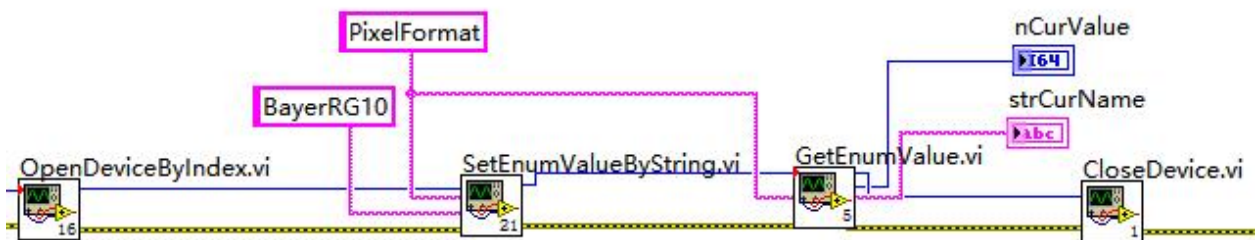
枚举型：

相关接口：

SetEnumValueByString.vi //设置

GetEnumValue.vi //读取

代码样例：



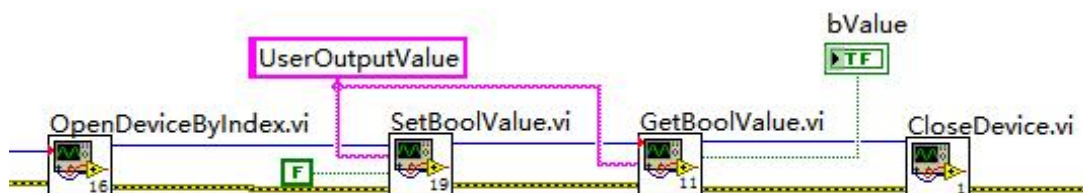
布尔型：

相关接口：

SetBoolValue.vi //设置

GetBoolValue.vi //读取

代码样例：



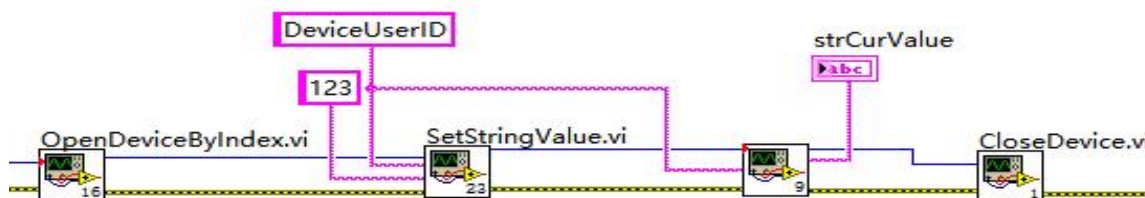
字符串型：

相关接口：

SetStringValue.vi //设置

GetStringValue.vi //读取

代码样例：

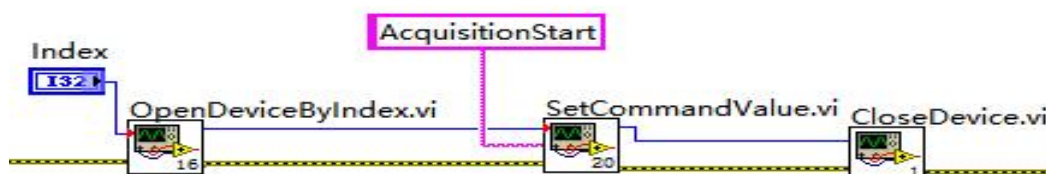


命令类型：

相关接口：

SetCommandValue.vi //设置

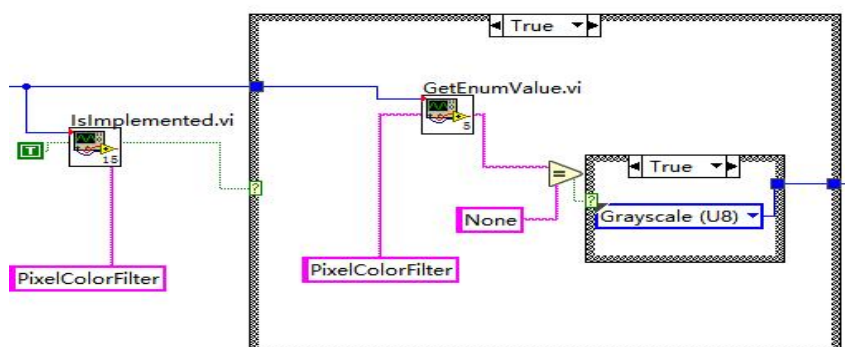
代码样例：



建议用户在操作属性参数前，先查询属性的功能是否实现：

查询属性功能是否实现的相关接口：

IsImplemented.vi //是否支持



2.2.6. 错误处理

LabVIEW 的错误信息由错误输入(error in)以及错误输出(error out)两个错误簇控件来表示, 该簇是由一个表示状态的布尔变量(status)、一个表示错误代码的 32 位整型变量(code)以及一个说明错误原因的字符串变量(source)组成的, 错误簇用于传递 LabVIEW 代码运行期间的出错信息。

Vi 模块运行失败时产生错误的 vi 信息填写到错误输出(error out)的 source 中, GxI API 产生的错误码填写到错误输出(error out)的 code 中, 对于重要的 vi 组件会将错误输出(error out)的 status 设置为 false, 如果添加了 Simple Error Handler VI 将会如图 2-3 所示:

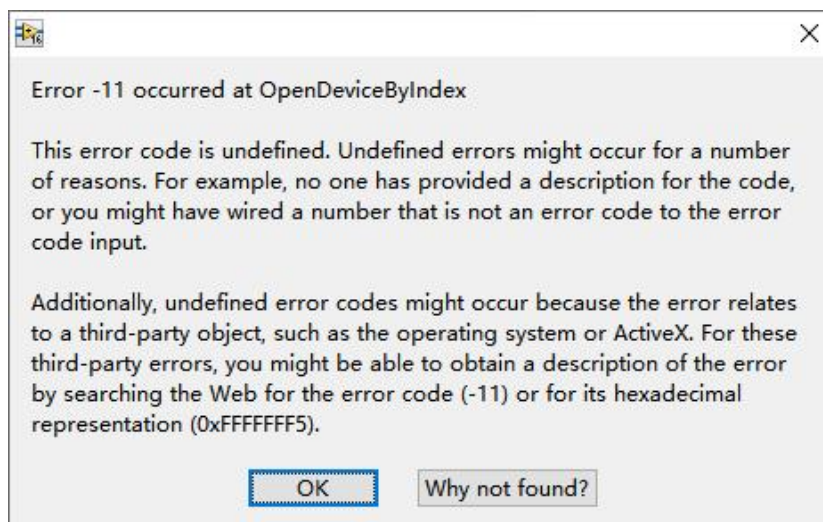


图 2-4 错误提示

3. 模块接口定义

3.1. InitLib.vi

意义：

初始化库资源

输入参数：

error in：错误输入

输出参数：

GX_STATUS：输出错误码

error out：错误输出

3.2. CloseLib.vi

意义：

释放库资源

输入参数：

error in：错误输入

输出参数：

GX_STATUS：输出错误码

error out：错误输出

3.3. UpdateAllDeviceList.vi

意义：

枚举设备（对于千兆网设备是子网枚举）

输入参数：

nTimeOut：枚举超时时间，单位 ms

error in：错误输入

输出参数：

punNumDevices：设备个数

GX_STATUS：输出错误码

error out：错误输出

3.4. OpenDeviceByIndex.vi

意义：

通过下标打开设备

输入参数：

nDeviceIndex：设备下标
error in：错误输入

输出参数：

phDevice：设备句柄
GX_STATUS：输出错误码
error out：错误输出

3.5. CloseDevice.vi

意义：

关闭设备

输入参数：

nInputHandle：设备句柄
error in：错误输入

输出参数：

GX_STATUS：输出错误码
error out：错误输出

3.6. IsImplemented.vi

意义：

查询当前属性控制是否支持读取或者写入的功能

输入参数：

nInputHandle：设备句柄
Read Data：是否是读取属性
strName： 属性名称
error in：错误输入

输出参数：

nOutputHandle：设备句柄
Result：是否支持
GX_STATUS：输出错误码
error out：错误输出

3.7. GetBoolValue.vi

意义：

获取布尔类型值

输入参数:

nInputHandle: 设备句柄

strName: 属性名称

error in: 错误输入

输出参数:

nOutputHandle: 设备句柄

bValue: Bool 类型值

GX_STATUS: 输出错误码

error out: 错误输出

3.8. SetBoolValue.vi

意义:

设置布尔类型值

输入参数:

nInputHandle: 设备句柄

strName: 属性名称

bValue: Bool 类型值

error in: 错误输入

输出参数:

nOutputHandle: 设备句柄

GX_STATUS: 输出错误码

error out: 错误输出

3.9. GetIntValue.vi

意义:

获取整型类型值

输入参数:

nInputHandle: 设备句柄

strName: 属性名称

error in: 错误输入

输出参数:

nOutputHandle: 设备句柄

nCurValue: Int 类型值

pstIntValue: GX_INT_VALUE 结构体, 详见 GxI API.h

GX_STATUS: 输出错误码

error out: 错误输出

3.10. SetIntValue.vi

意义:

设置整型类型值

输入参数:

nInputHandle: 设备句柄

strName: 属性名称

nValue: Int 类型值

error in: 错误输入

输出参数:

nOutputHandle: 设备句柄

GX_STATUS: 输出错误码

error out: 错误输出

3.11. GetFloatValue.vi

意义:

获取浮点型类型值

输入参数:

nInputHandle: 设备句柄

strName: 属性名称

error in: 错误输入

输出参数:

nOutputHandle: 设备句柄

dCurValue: float 类型值

pstFloatValue: GX_FLOAT_VALUE 结构体, 详见 GxI API.h

GX_STATUS: 输出错误码

error out: 错误输出

3.12. SetFloatValue.vi

意义:

设置浮点型类型值

输入参数：

nInputHandle：设备句柄
 strName： 属性名称
 fValue：float 类型值
 error in： 错误输入

输出参数：

nOutputHandle：设备句柄
 GX_STATUS：输出错误码
 error out： 错误输出

3.13. GetEnumValue.vi

意义：

获取枚举型类型值

输入参数：

nInputHandle：设备句柄
 strName： 属性名称
 error in： 错误输入

输出参数：

nOutputHandle：设备句柄
 nCurValue：enum 当前值
 strCurName：enum 当前值字符串名称
 pstIntValue： GX_ENUM_VALUE 结构体，详见 GxIAP.h
 GX_STATUS：输出错误码
 error out： 错误输出

3.14. SetEnumValueByString.vi

意义：

设置枚举型类型值

输入参数：

nInputHandle：设备句柄
 strName： 属性名称
 strValue：枚举名称
 error in： 错误输入

输出参数：

nOutputHandle: 设备句柄

GX_STATUS: 输出错误码

error out: 错误输出

3.15. GetStringValue.vi

意义:

获取字符串型类型值

输入参数:

nInputHandle: 设备句柄

strName: 属性名称

error in: 错误输入

输出参数:

nOutputHandle: 设备句柄

strCurValue: string 当前值

pstStringValue: GX_STRING_VALUE 结构体, 详见 GxIAP.h

GX_STATUS: 输出错误码

error out: 错误输出

3.16. SetStringValue.vi

意义:

设置字符串型类型值

输入参数:

nInputHandle: 设备句柄

strName: 属性名称

strValue: 设置字符串值

error in: 错误输入

输出参数:

nOutputHandle: 设备句柄

GX_STATUS: 输出错误码

error out: 错误输出

3.17. SetCommandValue.vi

意义:

执行命令

输入参数：

nInputHandle：设备句柄

strName： 命令名称

error in： 错误输入

输出参数：

nOutputHandle：设备句柄

GX_STATUS：输出错误码

error out：错误输出

3.18. StartGrab.vi

意义：

开始采集

输入参数：

nInputHandle：设备句柄

error in： 错误输入

输出参数：

nOutputHandle：设备句柄

GX_STATUS：输出错误码

error out：错误输出

3.19. StopGrab.vi

意义：

停止采集

输入参数：

nInputHandle：设备句柄

error in： 错误输入

输出参数：

nOutputHandle：设备句柄

GX_STATUS：输出错误码

error out：错误输出

3.20. GrabImageDisplay.vi

意义：

显示图像

输入参数：

pInputBuffer: 图像 Buffer
 nPixelFormat: 数据格式
 nWidth: 宽度
 nHeight: 高度
 Image Src: 为存储图像数据开辟空间
 error in: 错误输入

输出参数：

Image out: 输出图像为 IMAQImage 格式
 error out: 错误输出

3.21. DQBuf.vi

意义：

采集之后可通过此接口获取一张图片

输入参数：

nInputHandle: 设备句柄
 nTimeOut: 超时时间
 error in: 错误输入

输出参数：

nOutputHandle: 设备句柄
 ImgBuf: 图像 Buffer
 ppFrameBuffer: PGX_FRAME_BUFFER 结构体内容，详见 GxIAPI.h
 GX_STATUS: 输出错误码
 error out: 错误输出

3.22. QBuf.vi

意义：

在开始采集之后，通过此接口可以将图像数据 Buf 放回 GxIAPI 库，继续用于采集

输入参数：

nInputHandle: 设备句柄
 pBufferHandle: PGX_FRAME_BUFFER 结构体句柄（DQBuf 将输出），详见 GxIAPI.h
 error in: 错误输入

输出参数：

nOutputHandle: 设备句柄

GX_STATUS: 输出错误码

error out: 错误输出

3.23. ExportConfigFile.vi

意义:

导出相机当前参数到配置文件 (ANSI 接口)

输入参数:

nInputHandle: 设备句柄

pszFilePath: 配置文件输出路径

error in: 错误输入

输出参数:

nOutputHandle: 设备句柄

GX_STATUS: 输出错误码

error out: 错误输出

3.24. ImportConfigFile.vi

意义:

将配置文件中参数导入到相机 (ANSI 接口)

输入参数:

nInputHandle: 设备句柄

pszFilePath: 配置文件路径

bVerify: 如果此值为 true, 所有导入进去的值将会被读出进行校验是否一致

error in: 错误输入

输出参数:

nOutputHandle: 设备句柄

GX_STATUS: 输出错误码

error out: 错误输出

3.25. GetNodeAccessMode.vi

意义:

获取节点的读写属性

输入参数:

nInputHandle: 设备句柄

strName: 节点名称

error in: 错误输入

输出参数:

nOutputHandle: 设备句柄

pemAccessMode: 节点读写属性

GX_STATUS: 输出错误码

error out: 错误输出

3.26. ImageFormatConvert.vi

意义:

图像格式转换, 支持的转换格式详见函数 DxImageFormatConvert 注释, DxImageProc.h 头文件

输入参数:

pInputBuffer: 图像 Buffer

nWidth: 宽度

nHeight: 高度

nSrcPixelFormat: 源图像格式

nDstPixelFormat: 目标图像格式

nValidBits: 有效位

bFlip: 图像翻转与否, true: 翻转 false: 不翻转

emConvertType: Bayer 格式转换 RGB 类型, DX_BAYER_CONVERT_TYPE 结构体, 详见 DxImageProc.h 头文件

error in: 错误输入

输出参数:

nOutputBuffer: 输出 Buffer

nOutPutBufferSize: 输出 buffer 大小

error out: 错误输出

3.27. GetDataStreamHandleFromDev.vi

意义：

获取设备流通道句柄

输入参数：

nInputHandle：设备句柄

DSIndex：设备流序号，序号从 1 开始

error in：错误输入

输出参数：

phDS：设备流句柄

GX_STATUS：输出错误码

error out：错误输出

3.28. FlushQueue.vi

意义：

清空采集输出队列

输入参数：

nInputHandle：设备句柄

error in：错误输入

输出参数：

nOutputHandle：设备句柄

GX_STATUS：输出错误码

error out：错误输出

3.29. GetOptimalPacketSize.vi

意义：

获取最优包长值

输入参数：

nInputHandle：设备句柄

error in：错误输入

输出参数：

nOutputHandle：设备句柄

punPacketSize：最优包长值

GX_STATUS：输出错误码

error out: 错误输出

3.30. GetLastError.vi

意义:

获取程序最后的错误描述信息

输入参数:

error in: 错误输入

输出参数:

pErrorCode: 返回最后的错误码

pszErrText: 返回错误信息缓冲区地址

pSize: 错误信息缓冲区地址大小

GX_STATUS: 输出错误码

3.31. GetInterfaceHandle.vi

意义:

获取次序为 Index 的 Interface Handle

输入参数:

nIndex: Interface 次序, 从 1 开始

error in: 错误输入

输出参数:

phIF: 返回 Interface 句柄

GX_STATUS: 输出错误码

error out: 错误输出

3.32. GetU3VDeviceInfo.vi

意义:

获取次序为 Index 的设备信息

输入参数:

nIndex: 设备次序, 从 1 开始

error in: 错误输入

输出参数:

pstDeviceInfo: 返回设备信息, GX_DEVICE_INFO 结构体, 详见 GxI API.h

GX_STATUS: 输出错误码

error out: 错误输出

3.1. GetGEVDeviceInfo.vi

意义：

获取次序为 Index 的设备信息

输入参数：

nIndex：设备次序，从 1 开始

error in：错误输入

输出参数：

pstDeviceInfo：返回设备信息，GX_DEVICE_INFO 结构体，详见 GxI API.h

GX_STATUS：输出错误码

error out：错误输出

3.1. GetUSBDeviceInfo.vi

意义：

获取次序为 Index 的设备信息

输入参数：

nIndex：设备次序，从 1 开始

error in：错误输入

输出参数：

pstDeviceInfo：返回设备信息，GX_DEVICE_INFO 结构体，详见 GxI API.h

GX_STATUS：输出错误码

error out：错误输出

3.1. GetCXPDeviceInfo.vi

意义：

获取次序为 Index 的设备信息

输入参数：

nIndex：设备次序，从 1 开始

error in：错误输入

输出参数：

pstDeviceInfo：返回设备信息，GX_DEVICE_INFO 结构体，详见 GxI API.h

GX_STATUS：输出错误码

error out：错误输出

4. 常见问题解答

问题 1: LabVIEW 加载 VI 过程中, 会弹出搜索 dll 界面。

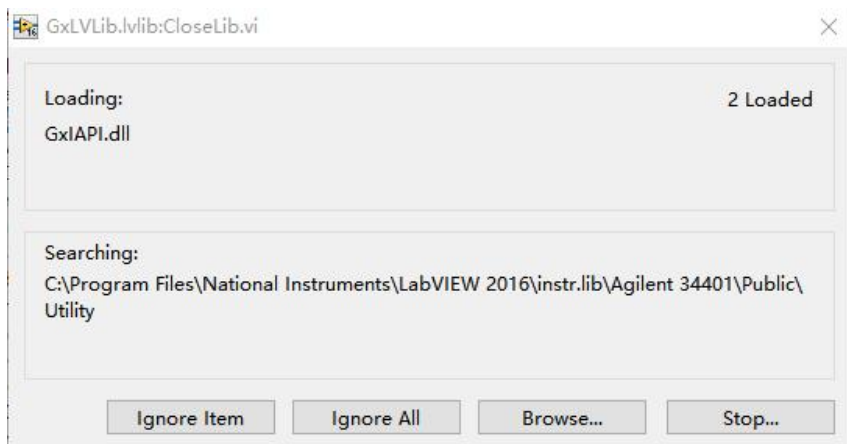


图 4-1 加载 VI 界面

解决方法:

该界面会自动搜索 VI 调用到的 GxIAPL.dll, 如果无法搜索到, 请用户手动点击浏览添加 SDK 中的 GxIAPL.dll 路径即可。

问题 2: LabVIEW 运行 VI 过程中, 提示以下信息。

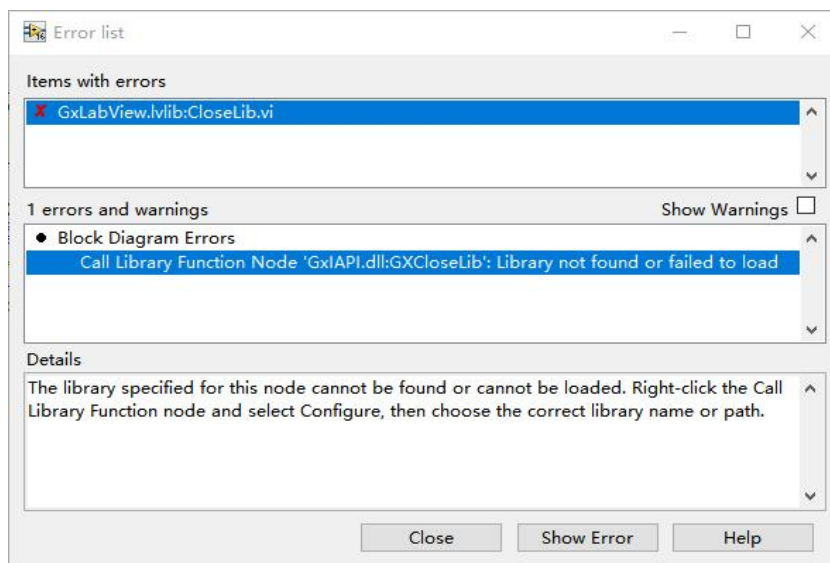


图 4-2 错误界面

该界面时因为自动搜索路径未找到 LabVIEW 对应版本 (x64 或 x86) 的 dll, 未找到同类版本, 找到其他版本 GxIAPL.dll 时候出现该错误。解决方法添加响应 GxIAPL.dll

问题 3: LabVIEW 运行 vi 时, 出现报错提示为 IMAQ 相关错误。

解决方案:

在 LabVIEW 官网下载 IMAQ VISION 相关工具包。

5. 版本说明

序号	修订版本号	所做改动	发布日期
1	V1.0.0	初始发布	2025-07-15